MENDSPEECH REVISED LEARNING SYSTEM

MendSpeech Complete 56 Day Daily Plan

Every day specifies what to learn, what to build, what to measure, the required output, compute choice, and the completion check.

> Eight weeks is the minimum calendar structure. Expect eight to ten weeks in practice. Do not move on from a day whose completion check is not satisfied.

How to Use This Plan

- Work six focused days each week and use Day 7 of each week for consolidation.
- Use the listed compute target as a default, not a requirement to waste GPU time.
- Keep experiment variables controlled and record model revision, data revision, hardware, software versions, seed, and settings.
- Every day must leave behind named evidence.
- Week 8 compares the practical cascaded MendSpeech baseline with direct audio repair so the final report acknowledges the text bottleneck instead of ignoring it.

Week 1: Audio, Degradation, and Measurement Foundations

> Build the audio laboratory and make SpeechDamageBench a deterministic standalone package.

Day	Focus	Minimum evidence	Compute
Day 1	Waveforms, sampling, and the MendSpeech baseline	• Compare 8 kHz, 16 kHz, 24 kHz, and 48 kHz versions by listening and plotting. • Measure duration, RMS energy, and file size for each version.	Local CPU
Day 2	Fourier intuition and STFT	• Hold audio constant and change one STFT setting at a time. • Write what phonetic or transient detail becomes easier or harder to see.	Local CPU
Day 3	Mel scale and log Mel features	• Change Mel bin count and compare visual structure and compute size. • Verify consistent feature shapes for different utterance lengths.	Local CPU
Day 4	Build SpeechDamageBench as a standalone package	• Generate mild, medium, and severe examples from the same clean sentence. • Reproduce the exact same damaged waveform from the same seed. • Change only the seed and verify that the corruption changes while all configured parameters remain fixed.	Local CPU
Day 5	Objective audio measurements	• Run all corruption levels on at least ten clips. • Look for cases where a metric disagrees with your listening judgment.	Local CPU
Day 6	Build the first MendSpeech audio console	• Test with three speakers and several corruption types. • Write down usability problems that would block later live debugging.	Local CPU
Day 7	Review, explain, and freeze Week 1	• From a blank notebook, recreate one corruption and one log Mel plot without copying previous cells.	Local CPU

DAY 1

Waveforms, sampling, and the MendSpeech baseline

Compute
Local CPU

LEARN

- Waveform amplitude and time axes.
- Sampling rate, Nyquist intuition, bit depth, mono versus stereo.
- Why speech systems often standardize to 16 kHz.
- Duration, peak amplitude, RMS energy, and clipping.

BUILD IN MENDSPEECH

- Create the repository and a minimal audio loader.
- Record or collect five clean speech clips with consent.
- Normalize all clips to a consistent sample rate and mono format.

EXPERIMENT AND MEASURE

- Compare 8 kHz, 16 kHz, 24 kHz, and 48 kHz versions by listening and plotting.
- Measure duration, RMS energy, and file size for each version.

REQUIRED OUTPUT

- notebooks/day01_waveform.ipynb
- data/clean_manifest.csv
- docs/audio_baseline_notes.md

> **Completion check:** You can explain what is lost when sample rate is reduced and can reproduce the same preprocessing from code.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

DAY 2

Fourier intuition and STFT

Compute
Local CPU

LEARN

- Frequency, phase, harmonics, and spectral energy.
- Fourier transform intuition without memorizing derivations.
- STFT frames, window length, hop length, overlap.
- Tradeoff between time resolution and frequency resolution.

BUILD IN MENDSPEECH

- Implement STFT visualization with PyTorch or TorchAudio.
- Plot the same utterance with several window and hop settings.

EXPERIMENT AND MEASURE

- Hold audio constant and change one STFT setting at a time.
- Write what phonetic or transient detail becomes easier or harder to see.

REQUIRED OUTPUT

- notebooks/day02_stft.ipynb
- results/stft_parameter_grid.png

Completion check: You can choose a reasonable frame and hop configuration and explain why.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

DAY 3

Mel scale and log Mel features

Compute
Local CPU

LEARN

- Human frequency perception and the Mel scale.
- Mel filterbanks and log compression.
- Number of Mel bins and dynamic range.
- Normalization of acoustic features.

BUILD IN MENDSPEECH

- Implement or inspect a log Mel feature pipeline.
- Build a function that returns features plus metadata needed for reproducibility.

EXPERIMENT AND MEASURE

- Change Mel bin count and compare visual structure and compute size.
- Verify consistent feature shapes for different utterance lengths.

REQUIRED OUTPUT

- src/audio/features.py
- notebooks/day03_logmel.ipynb
- tests/test_features.py

> **Completion check:** You can trace waveform to STFT to Mel filterbank to log Mel tensor.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

DAY 4

Build SpeechDamageBench as a standalone package

Compute
Local CPU

LEARN

- Deterministic corruption design and seed control.
- Additive noise, clipping, bandwidth limitation, dropouts, and reverberation.
- Why a benchmark should be reusable outside the main application.
- Versioned severity presets and manifest metadata.

BUILD IN MENDSPEECH

- Create an installable speechdamagebench package instead of burying corruptions inside MendSpeech.
- Implement noise, clipping, bandwidth reduction, dropout, and simple reverberation modules.
- Add a seed controlled configuration object and a small command line entry point.
- Record corruption name, severity, seed, parameters, and clean source id for every output.

EXPERIMENT AND MEASURE

- Generate mild, medium, and severe examples from the same clean sentence.
- Reproduce the exact same damaged waveform from the same seed.
- Change only the seed and verify that the corruption changes while all configured parameters remain fixed.

REQUIRED OUTPUT

- speechdamagebench/audio_damage.py
- speechdamagebench/presets.py
- speechdamagebench/cli.py
- pyproject.toml
- tests/test_determinism.py
- configs/damage_levels.yaml

> **Completion check:** Another project can install SpeechDamageBench and regenerate the same damaged clip from a manifest entry.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

DAY 5

Objective audio measurements

Compute
Local CPU

LEARN

- RMS and peak level.
- Simple SNR calculation when the clean reference is known.
- Spectral distance intuition.
- Why perceptual speech quality is not fully captured by one scalar metric.

BUILD IN MENDSPEECH

- Add baseline metrics for clean versus corrupted pairs.
- Store results in a tidy CSV schema with clip id, corruption, severity, seed, and measurements.

EXPERIMENT AND MEASURE

- Run all corruption levels on at least ten clips.
- Look for cases where a metric disagrees with your listening judgment.

REQUIRED OUTPUT

- src/metrics/audio_metrics.py
- results/week1_damage_metrics.csv
- docs/metric_limitations.md

> **Completion check:** You can explain what each metric says and what it fails to say.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

DAY 6

Build the first MendSpeech audio console

Compute
Local CPU

LEARN

- Audio playback in a lightweight interface.
- Waveform and spectrogram synchronization.
- Before and after comparison design.

BUILD IN MENDSPEECH

- Build a local page or notebook dashboard with clean and damaged playback.
- Add corruption controls and immediately regenerate the damaged clip.
- Display waveform, spectrogram, and basic measurements.

EXPERIMENT AND MEASURE

- Test with three speakers and several corruption types.
- Write down usability problems that would block later live debugging.

REQUIRED OUTPUT

- app/audio_lab.py
- screenshots/week1_audio_console.png

> **Completion check:** Another person can open the tool, damage an utterance, and understand the visual change without reading your code.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

DAY 7

Review, explain, and freeze Week 1

Compute
Local CPU

LEARN

- Review waveform, STFT, Mel features, SNR, clipping, dropouts, and reverberation.

BUILD IN MENDSPEECH

- Clean repository structure.
- Freeze SpeechDamageBench v0.1 severity presets and at least ten clean reference clips.
- Tag the benchmark package schema and add a minimal usage example independent of MendSpeech.

EXPERIMENT AND MEASURE

- From a blank notebook, recreate one corruption and one log Mel plot without copying previous cells.

REQUIRED OUTPUT

- reports/week1_audio_foundations.md
- data/week1_reference_manifest.csv
- speechdamagebench/README.md
- speechdamagebench/VERSION

> **Completion check:** You can teach the complete path from clean waveform to controlled corruption and feature tensor.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- PyTorch and TorchAudio audio processing documentation
- A practical digital signal processing reference for STFT and filterbanks

Week 2: ASR, CTC, Confidence, and Repair Localization

> Build the recognition and uncertainty layer, plus a reusable Modal execution path.

Day	Focus	Minimum evidence	Compute
Day 8	Frame sequence to transcript	• Compare clean and corrupted transcripts on the exact same utterances.	Modal L4 optional, CPU acceptable for small runs
Day 9	CTC from first principles	• Enumerate several legal paths for a tiny target word. • Break your decoder deliberately with repeated letters and fix it.	Local CPU
Day 10	WER, CER, and error taxonomy	• Score clean versus every SpeechDamageBench severity. • Find which corruption type causes deletion errors fastest.	Local CPU
Day 11	Token confidence and uncertainty	• Compare confidence on clean, noisy, clipped, and dropout audio. • Find confident but wrong examples and document them.	Modal L4 recommended
Day 12	Time alignment and uncertain spans	• Inject known 100 ms and 250 ms dropouts and test whether uncertainty overlaps them.	Modal L4 recommended
Day 13	Define selective repair policy v0	• Sweep confidence thresholds on SpeechDamageBench. • Measure percentage of audio selected for repair and overlap with known damaged spans.	Local CPU after ASR outputs are cached
Day 14	Week 2 integration and review	• Run at least twenty corrupted utterances and manually inspect policy errors.	Modal L4 recommended

DAY 8

Frame sequence to transcript

Compute
Modal L4 optional, CPU acceptable for small runs

LEARN

- Why acoustic frames outnumber output tokens.
- Encoder outputs, vocabulary logits, and decoding.
- CTC versus transducer versus attention decoder at a high level.

BUILD IN MENDSPEECH

- Run a pretrained ASR model on clean and damaged SpeechDamageBench clips.
- Store transcript, token outputs if available, and timing metadata.
- Add a reusable Modal entry point so the same command can run ASR experiments on an L4 without editing deployment code each day.

EXPERIMENT AND MEASURE

- Compare clean and corrupted transcripts on the exact same utterances.

REQUIRED OUTPUT

- src/asr/baseline.py
- infra/modal_asr.py
- results/day08_baseline_transcripts.csv

> **Completion check:** You can draw the path from features to encoder states to token probabilities to text, and launch the same baseline locally or on Modal with a documented command.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

DAY 9

CTC from first principles

Compute
Local CPU

LEARN

- CTC blank symbol.
- Repeated labels and collapse operation.
- Why many frame paths map to one transcript.
- Conditional independence assumption and its consequence.

BUILD IN MENDSPEECH

- Implement CTC collapse yourself without a library decoder.
- Create hand written alignment examples and unit tests.

EXPERIMENT AND MEASURE

- Enumerate several legal paths for a tiny target word.
- Break your decoder deliberately with repeated letters and fix it.

REQUIRED OUTPUT

- src/asr/ctc_decode.py
- tests/test_ctc_decode.py
- docs/ctc_explained.md

> **Completion check:** You can explain why a blank is needed and correctly decode repeated characters.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

DAY 10

WER, CER, and error taxonomy

Compute
Local CPU

LEARN

- Word error rate: substitutions, deletions, insertions.
- Character error rate and when it helps.
- Why WER alone hides error severity.

BUILD IN MENDSPEECH

- Implement or verify WER and CER calculations.
- Add an error analyzer that labels substitution, deletion, and insertion spans.

EXPERIMENT AND MEASURE

- Score clean versus every SpeechDamageBench severity.
- Find which corruption type causes deletion errors fastest.

REQUIRED OUTPUT

- src/metrics/wer.py
- results/day10_wer_by_damage.csv
- results/day10_error_types.csv

Completion check: You can calculate WER by hand for a short example and explain each error.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

DAY 11

Token confidence and uncertainty

Compute
Modal L4 recommended

LEARN

- Softmax confidence and why it can be miscalibrated.
- Frame confidence versus token confidence versus word confidence.
- Entropy as an uncertainty signal.
- Confidence calibration intuition.

BUILD IN MENDSPEECH

- Extract confidence or approximate it from model outputs.
- Create a word level confidence timeline aligned to the transcript.

EXPERIMENT AND MEASURE

- Compare confidence on clean, noisy, clipped, and dropout audio.
- Find confident but wrong examples and document them.

REQUIRED OUTPUT

- src/asr/confidence.py
- results/day11_confidence_cases.csv
- docs/confidence_failure_modes.md

Completion check: You understand why low confidence can be useful but cannot be treated as truth.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

DAY 12

Time alignment and uncertain spans

Compute
Modal L4 recommended

LEARN

- Frame time conversion.
- Token timestamps and word timestamps.
- Alignment boundaries around corrupted regions.

BUILD IN MENDSPEECH

- Map low confidence tokens back to audio time spans.
- Overlay uncertain intervals on waveform and spectrogram.

EXPERIMENT AND MEASURE

- Inject known 100 ms and 250 ms dropouts and test whether uncertainty overlaps them.

REQUIRED OUTPUT

- src/asr/alignment.py
- app/uncertainty_overlay.py
- results/day12_overlap_metrics.csv

> **Completion check:** The UI can highlight an uncertain audio interval and show the associated word or token.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

DAY 13

Define selective repair policy v0

Compute
Local CPU after ASR outputs are cached

LEARN

- Threshold policies.
- Hysteresis to avoid rapid toggling.
- Minimum repair span and padding.
- False repair versus missed repair tradeoff.

BUILD IN MENDSPEECH

- Implement Preserve, Balanced, and Rescue policies.
- Each policy returns preserve spans and repair spans from confidence plus timing.

EXPERIMENT AND MEASURE

- Sweep confidence thresholds on SpeechDamageBench.
- Measure percentage of audio selected for repair and overlap with known damaged spans.

REQUIRED OUTPUT

- src/controller/policy.py
- configs/repair_modes.yaml
- results/day13_policy_sweep.csv

Completion check: You can explain the cost of repairing too much and repairing too little.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

DAY 14

Week 2 integration and review

Compute
Modal L4 recommended

LEARN

- Review CTC, WER, confidence, timestamp alignment, and repair decisions.

BUILD IN MENDSPEECH

- Pipeline: damaged audio to transcript to confidence to highlighted repair spans.
- Add clean JSON output for every run.
- Verify the Modal wrapper records model revision, GPU type, software versions, and run id automatically.

EXPERIMENT AND MEASURE

- Run at least twenty corrupted utterances and manually inspect policy errors.

REQUIRED OUTPUT

- app/mendspeech_v0.py
- infra/modal_asr.py
- results/week2_casebook.md
- reports/week2_asr_uncertainty.md

> **Completion check:** A user can see what the ASR heard and which exact intervals MendSpeech wants to preserve or repair.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

Week 3: Conformer From First Principles

> Implement the core encoder pieces so model behavior is not a black box.

Day	Focus	Minimum evidence	Compute
Day 15	Attention for speech sequences	• Change sequence length and measure forward time and memory.	Local CPU, L4 optional for scaling
Day 16	Conformer convolution module	• Feed synthetic impulses and inspect how local information spreads.	Local CPU
Day 17	Macaron feed forward and residual scaling	• Compare output statistics with and without residual scaling.	Local CPU
Day 18	Assemble one Conformer block	• Run forward and backward tests on several sequence lengths. • Intentionally remove one residual path and compare training stability on a toy task.	Local CPU
Day 19	Build a tiny Conformer encoder	• Track tensor shape through every layer on real speech. • Profile increasing depth.	Local CPU, L4 optional
Day 20	Compare your block with a production	• Choose one difference and reproduce its effect on a small benchmark if feasible.	Local CPU
Day 21	Week 3 architecture review	• Give yourself a ten minute whiteboard explanation from waveform features through one Conformer block.	Local CPU

DAY 15

Attention for speech sequences

Compute
Local CPU, L4 optional for scaling

LEARN

- Query, key, value projections.
- Scaled dot product attention.
- Attention masks.
- Sequence length cost.

BUILD IN MENDSPEECH

- Implement single head attention and then multi head attention in PyTorch.
- Add shape assertions and gradient tests.

EXPERIMENT AND MEASURE

- Change sequence length and measure forward time and memory.

REQUIRED OUTPUT

- src/models/attention.py
- tests/test_attention.py
- results/day15_attention_scaling.csv

Completion check: You can derive every major tensor shape and explain quadratic sequence cost.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Conformer primary paper
- A mature Conformer implementation such as NVIDIA NeMo

DAY 16

Conformer convolution module

Compute
Local CPU

LEARN

- Pointwise convolution.
- GLU gating.
- Depthwise convolution.
- Batch normalization and activation.
- Why local patterns matter in speech.

BUILD IN MENDSPEECH

- Implement a Conformer style convolution module.
- Test causality assumptions and receptive field.

EXPERIMENT AND MEASURE

- Feed synthetic impulses and inspect how local information spreads.

REQUIRED OUTPUT

- src/models/conformer_conv.py
- tests/test_conformer_conv.py
- notebooks/day16_receptive_field.ipynb

> **Completion check:** You can explain why depthwise convolution is computationally attractive and what local context it captures.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Conformer primary paper
- A mature Conformer implementation such as NVIDIA NeMo

DAY 17

Macaron feed forward and residual scaling

Compute
Local CPU

LEARN

- Feed forward expansion.
- Swish or SiLU activation.
- Dropout.
- Half step residual weighting in Conformer.

BUILD IN MENDSPEECH

- Implement the feed forward module and residual wrapper.
- Add numerical tests for shape and gradient flow.

EXPERIMENT AND MEASURE

- Compare output statistics with and without residual scaling.

REQUIRED OUTPUT

- src/models/conformer_ffn.py
- tests/test_conformer_ffn.py

> **Completion check:** You can explain the ordering of the Conformer block without memorizing a diagram.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Conformer primary paper
- A mature Conformer implementation such as NVIDIA NeMo

DAY 18

Assemble one Conformer block

Compute
Local CPU

LEARN

- Macaron structure.
- Layer normalization placement.
- Attention plus convolution interaction.

BUILD IN MENDSPEECH

- Assemble feed forward, attention, convolution, second feed forward, and final normalization.
- Match expected input and output shapes.

EXPERIMENT AND MEASURE

- Run forward and backward tests on several sequence lengths.
- Intentionally remove one residual path and compare training stability on a toy task.

REQUIRED OUTPUT

- src/models/conformer_block.py
- tests/test_conformer_block.py
- docs/conformer_block_walkthrough.md

> **Completion check:** You can point to every operation and say why it exists.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Conformer primary paper
- A mature Conformer implementation such as NVIDIA NeMo

DAY 19

Build a tiny Conformer encoder

Compute
Local CPU, L4 optional

LEARN

- Input projection.
- Stacked blocks.
- Mask propagation.
- Temporal dimensions.

BUILD IN MENDSPEECH

- Build a small encoder around your blocks.
- Connect log Mel features to the encoder.

EXPERIMENT AND MEASURE

- Track tensor shape through every layer on real speech.
- Profile increasing depth.

REQUIRED OUTPUT

- src/models/tiny_conformer.py
- results/day19_shape_trace.md

Completion check: A real log Mel tensor can pass through your encoder and produce valid gradients.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Conformer primary paper
- A mature Conformer implementation such as NVIDIA NeMo

DAY 20

Compare your block with a production

Compute
Local CPU

LEARN

- Read the original Conformer paper sections relevant to block design.
- Inspect a mature implementation such as NeMo.
- Identify differences caused by engineering and efficiency.

BUILD IN MENDSPEECH

- Create an annotated comparison table: your component, paper definition, production implementation.

EXPERIMENT AND MEASURE

- Choose one difference and reproduce its effect on a small benchmark if feasible.

REQUIRED OUTPUT

- docs/day20_implementation_comparison.md

> **Completion check:** You can read production Conformer code and orient yourself without treating it as magic.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Conformer primary paper
- A mature Conformer implementation such as NVIDIA NeMo

DAY 21

Week 3 architecture review

Compute
Local CPU

LEARN

- Review attention, convolution, feed forward, normalization, residual paths, and sequence cost.

BUILD IN MENDSPEECH

- Add an architecture inspector page to MendSpeech showing encoder stage shapes and context assumptions.

EXPERIMENT AND MEASURE

- Give yourself a ten minute whiteboard explanation from waveform features through one Conformer block.

REQUIRED OUTPUT

- app/encoder_inspector.py
- reports/week3_conformer.md

> **Completion check:** You can explain which parts are local, which are global, and which become problematic for streaming.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Conformer primary paper
- A mature Conformer implementation such as NVIDIA NeMo

Week 4: FastConformer and Efficient Encoder Behavior

> Measure why FastConformer is efficient and freeze a reproducible baseline.

Day	Focus	Minimum evidence	Compute
Day 22	Why FastConformer exists	• Estimate attention matrix size before and after aggressive temporal subsampling.	Local CPU
Day 23	Temporal subsampling experiment	• Compare 2x, 4x, and 8x temporal reduction on tensor length, runtime, and rough output behavior.	Modal L4 useful
Day 24	Pretrained FastConformer baseline	• Benchmark WER, latency, and GPU memory by damage type.	Modal L4
Day 25	Context and attention limits	• If supported, compare at least two context settings on the same subset.	Modal L4
Day 26	Efficiency benchmark harness	• Run repeated inference and calculate variance. • Detect and discard obviously invalid cold start comparisons.	Modal L4
Day 27	FastConformer failure casebook	• Look for systematic error patterns rather than isolated anecdotes.	Modal L4
Day 28	Week 4 integration	• Run the same ten reference clips through the full Week 2 uncertainty policy using FastConformer.	Modal L4

DAY 22

Why FastConformer exists

Compute
Local CPU

LEARN

- Sequence length as an attention cost driver.
- Subsampling before expensive encoder blocks.
- Depthwise separable convolution.
- Local and limited context attention.

BUILD IN MENDSPEECH

- Read the FastConformer paper with a comparison checklist.
- Write a diagram showing what changes relative to Conformer.

EXPERIMENT AND MEASURE

- Estimate attention matrix size before and after aggressive temporal subsampling.

REQUIRED OUTPUT

- docs/day22_fastconformer_notes.md
- results/day22_compute_estimates.csv

> **Completion check:** You can explain FastConformer as a set of concrete efficiency choices, not just a faster model name.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 23

Temporal subsampling experiment

Compute
Modal L4 useful

LEARN

- Convolutional subsampling.
- Temporal resolution.
- Information loss versus compute reduction.

BUILD IN MENDSPEECH

- Implement a small subsampling front end or isolate one from a framework.
- Track frames per second before and after each stage.

EXPERIMENT AND MEASURE

- Compare 2x, 4x, and 8x temporal reduction on tensor length, runtime, and rough output behavior.

REQUIRED OUTPUT

- src/models/subsampling.py
- results/day23_subsampling.csv

> **Completion check:** You can quantify how subsampling changes sequence length and downstream attention cost.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 24

Pretrained FastConformer baseline

Compute
Modal L4

LEARN

- Model checkpoint loading.
- Tokenizer and decoder configuration.
- Batch versus single utterance inference.

BUILD IN MENDSPEECH

- Run a current NeMo FastConformer checkpoint on your clean and damaged sets.
- Record model revision and all inference settings.

EXPERIMENT AND MEASURE

- Benchmark WER, latency, and GPU memory by damage type.

REQUIRED OUTPUT

- src/asr/fastconformer_runner.py
- results/day24_fastconformer_baseline.csv
- configs/model_baseline.yaml

Completion check: You have a reproducible baseline with model, data, hardware, and settings fixed.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 25

Context and attention limits

Compute
Modal L4

LEARN

- Full context attention.
- Limited context attention.
- Left and right context.
- Accuracy versus latency intuition.

BUILD IN MENDSPEECH

- Inspect context settings in the model configuration.
- Create a visual timeline explaining visible past and future context.

EXPERIMENT AND MEASURE

- If supported, compare at least two context settings on the same subset.

REQUIRED OUTPUT

- docs/day25_context_timeline.md
- results/day25_context_compare.csv

> **Completion check:** You can explain exactly why future context creates algorithmic latency.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 26

Efficiency benchmark harness

Compute
Modal L4

LEARN

- Warmup runs.
- Synchronized GPU timing.
- Median and percentile latency.
- Real time factor.
- Peak memory.

BUILD IN MENDSPEECH

- Create one benchmark function used by every later experiment.
- Log environment and model metadata automatically.

EXPERIMENT AND MEASURE

- Run repeated inference and calculate variance.
- Detect and discard obviously invalid cold start comparisons.

REQUIRED OUTPUT

- src/bench/benchmark_asr.py
- src/bench/environment.py
- results/day26_repeatability.csv

> **Completion check:** Repeated runs produce stable enough numbers to support comparisons.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 27

FastConformer failure casebook

Compute
Modal L4

LEARN

- Error slicing by corruption type and severity.
- Short versus long utterance effects.
- Confidence versus error.

BUILD IN MENDSPEECH

- Build a casebook of at least fifteen interesting failures.
- Link each case to audio, transcript, confidence, and damage metadata.

EXPERIMENT AND MEASURE

- Look for systematic error patterns rather than isolated anecdotes.

REQUIRED OUTPUT

- results/fastconformer_failure_casebook.md

> **Completion check:** You can name at least three repeatable failure patterns and propose a testable reason for each.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

DAY 28

Week 4 integration

Compute
Modal L4

LEARN

- Review efficiency choices and baseline results.

BUILD IN MENDSPEECH

- Replace the generic ASR runner in MendSpeech with the reproducible FastConformer path.
- Expose latency, RTF, WER when reference text exists, and GPU memory in the research console.

EXPERIMENT AND MEASURE

- Run the same ten reference clips through the full Week 2 uncertainty policy using FastConformer.

REQUIRED OUTPUT

- app/mendspeech_v1_fastconformer.py
- reports/week4_fastconformer.md

> **Completion check:** MendSpeech now has a measured, inspectable FastConformer recognition core.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastConformer primary paper
- NVIDIA NeMo FastConformer model documentation

Week 5: Streaming, Cache Aware Inference, and Adaptive Context

> Turn the recognizer into a real time system and test uncertainty guided context spending.

Day	Focus	Minimum evidence	Compute
Day 29	Offline versus streaming ASR	• Compare offline transcript with naive chunk by chunk transcription.	Modal L4
Day 30	Buffered streaming	• Sweep buffer and stride settings. • Measure WER and latency tradeoffs.	Modal L4
Day 31	Cache aware streaming internals	• Compare buffered and cache aware inference on the same audio and same hardware.	Modal L4
Day 32	Lookahead ablation	• Plot WER versus latency and identify dominated operating points.	Modal L4
Day 33	Break the cache on purpose	• Measure WER changes around the reset point. • Inspect whether errors cluster near boundaries or propagate later.	Modal L4
Day 34	Adaptive context controller prototype	• Compare fixed fast, fixed accurate, and adaptive policies on a controlled subset.	Modal L4
Day 35	Week 5 live streaming milestone	• Record a short demo with clean and damaged speech. • Document remaining technical limitations honestly.	Modal L4

DAY 29

Offline versus streaming ASR

Compute
Modal L4

LEARN

- Audio chunks.
- Algorithmic latency.
- Partial hypotheses.
- Endpointing and finalization.

BUILD IN MENDSPEECH

- Create a chunk simulator that feeds audio incrementally.
- Log when each chunk becomes available and when text changes.

EXPERIMENT AND MEASURE

- Compare offline transcript with naive chunk by chunk transcription.

REQUIRED OUTPUT

- src/streaming/chunker.py
- results/day29_offline_vs_naive.csv

> **Completion check:** You can explain why naive chunking creates boundary errors and redundant compute.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

DAY 30

Buffered streaming

Compute
Modal L4

LEARN

- Overlapping windows.
- Buffer size.
- Stride.
- Repeated computation.

BUILD IN MENDSPEECH

- Implement or run buffered streaming with configurable overlap.
- Measure how much audio is recomputed.

EXPERIMENT AND MEASURE

- Sweep buffer and stride settings.
- Measure WER and latency tradeoffs.

REQUIRED OUTPUT

- src/streaming/buffered.py
- results/day30_buffered_sweep.csv

Completion check: You can quantify the compute waste caused by overlapping history.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

DAY 31

Cache aware streaming internals

Compute
Modal L4

LEARN

- Cached activations.
- Past context state.
- Streaming masks.
- Right context and lookahead.

BUILD IN MENDSPEECH

- Use NeMo cache aware streaming inference on a supported FastConformer checkpoint.
- Log cache related configuration and chunk boundaries.

EXPERIMENT AND MEASURE

- Compare buffered and cache aware inference on the same audio and same hardware.

REQUIRED OUTPUT

- src/streaming/cache_aware_runner.py
- results/day31_buffered_vs_cache.csv

> **Completion check:** You can explain what is cached, what is recomputed, and why cache aware inference can be more efficient.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

DAY 32

Lookahead ablation

Compute
Modal L4

LEARN

- Right context.
- Lookahead.
- Commit delay.
- WER and latency as competing objectives.

BUILD IN MENDSPEECH

- Run several supported lookahead settings with everything else fixed.
- Store per utterance and aggregate metrics.

EXPERIMENT AND MEASURE

- Plot WER versus latency and identify dominated operating points.

REQUIRED OUTPUT

- experiments/lookahead_ablation.py
- results/day32_lookahead.csv
- results/day32_pareto.png

> **Completion check:** You can defend a Balanced operating point using data rather than preference.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

DAY 33

Break the cache on purpose

Compute
Modal L4

LEARN

- State continuity.
- Chunk boundary dependencies.
- Cache reset and truncation.

BUILD IN MENDSPEECH

- Add controlled experiments that reset or shorten cache at selected boundaries.

EXPERIMENT AND MEASURE

- Measure WER changes around the reset point.
- Inspect whether errors cluster near boundaries or propagate later.

REQUIRED OUTPUT

- experiments/cache_break_test.py
- results/day33_cache_failures.md

Completion check: You can explain a concrete failure caused by incorrect state handling.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

DAY 34

Adaptive context controller prototype

Compute
Modal L4

LEARN

- Policy driven context selection.
- Confidence smoothing.
- Latency budget.
- Stability versus oscillation.

BUILD IN MENDSPEECH

- Implement a controller that classifies chunks as easy or uncertain.
- Map states to small or larger supported right context settings, even if the first prototype must simulate switching between runs.

EXPERIMENT AND MEASURE

- Compare fixed fast, fixed accurate, and adaptive policies on a controlled subset.

REQUIRED OUTPUT

- src/controller/adaptive_context.py
- results/day34_adaptive_context.csv

Completion check: You have a falsifiable first answer to whether uncertainty can guide context spending.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

DAY 35

Week 5 live streaming milestone

Compute
Modal L4

LEARN

- Review buffered streaming, cache aware inference, lookahead, cache failures, and adaptive context.

BUILD IN MENDSPEECH

- Connect microphone or simulated live audio to the streaming recognizer.
- Show partial text, confidence timeline, current context mode, and latency.

EXPERIMENT AND MEASURE

- Record a short demo with clean and damaged speech.
- Document remaining technical limitations honestly.

REQUIRED OUTPUT

- app/mendspeech_v2_streaming.py
- demos/week5_streaming_demo.mp4
- reports/week5_streaming.md

> **Completion check:** A person can speak and watch MendSpeech transcribe incrementally while exposing the state that drives repair decisions.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

Week 6: Robustness, Fine Tuning, RNNT, and Calibration

> Adapt the recognizer to damaged speech while learning training and calibration discipline.

Day	Focus	Minimum evidence	Compute
Day 36	Training pipeline anatomy	• Deliberately use a bad learning rate and record the failure signature.	Modal L4
Day 37	Build a robust fine tuning dataset	• Audit duplicate and speaker leakage.	Local CPU
Day 38	Fine tune for damaged speech robustness	• Compare base and adapted model on the frozen test set.	Modal L4, consider L40S only if memory blocks the planned experiment
Day 39	SpecAugment and augmentation ablation	• Compare no augmentation versus selected augmentation with the same seed and training budget.	Modal L4
Day 40	RNNT and streaming decoding	• Compare CTC and transducer outputs on selected difficult clips.	Modal L4
Day 41	Confidence calibration for repair decisions	• Compare raw and calibrated confidence if a simple method is feasible.	Modal L4 for logits, local CPU for analysis
Day 42	Week 6 robustness milestone	• Run one fixed benchmark suite and freeze results for Week 8 comparisons.	Modal L4

DAY 36

Training pipeline anatomy

Compute
Modal L4

LEARN

- Manifest format.
- Batching variable duration audio.
- Loss curves.
- Learning rate.
- Validation split.
- Checkpointing.

BUILD IN MENDSPEECH

- Create a tiny reproducible training configuration.
- Run a short smoke training job and verify loss decreases.

EXPERIMENT AND MEASURE

- Deliberately use a bad learning rate and record the failure signature.

REQUIRED OUTPUT

- configs/train_smoke.yaml
- results/day36_training_smoke.csv
- docs/training_debug_notes.md

Completion check: You can diagnose whether a run is learning, diverging, or overfitting from basic evidence.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- NVIDIA NeMo ASR training documentation
- RNNT primary references
- Calibration and reliability diagram references

DAY 37

Build a robust fine tuning dataset

Compute
Local CPU

LEARN

- Train, validation, test separation.
- Speaker leakage.
- Synthetic corruption sampling.
- Balanced severity distribution.

BUILD IN MENDSPEECH

- Create manifests that pair clean transcripts with corrupted audio.
- Keep a speaker separated test set frozen.

EXPERIMENT AND MEASURE

- Audit duplicate and speaker leakage.

REQUIRED OUTPUT

- data/train_manifest.jsonl
- data/val_manifest.jsonl
- data/test_manifest.jsonl
- reports/data_audit.md

Completion check: The evaluation set cannot accidentally appear in training through clean or corrupted duplicates.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- NVIDIA NeMo ASR training documentation
- RNNT primary references
- Calibration and reliability diagram references

DAY 38

Fine tune for damaged speech robustness

Compute
Modal L4, consider L40S only if memory blocks the planned experiment

LEARN

- Transfer learning.
- Frozen versus trainable layers.
- Mixed precision.
- Gradient accumulation.

BUILD IN MENDSPEECH

- Fine tune a manageable FastConformer or compatible ASR checkpoint on the robustness dataset.
- Save model, config, and training logs.

EXPERIMENT AND MEASURE

- Compare base and adapted model on the frozen test set.

REQUIRED OUTPUT

- training/finetune.py
- checkpoints/week6_best/
- results/day38_base_vs_adapted.csv

Completion check: You can state exactly what improved, what did not, and whether clean speech regressed.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- NVIDIA NeMo ASR training documentation
- RNNT primary references
- Calibration and reliability diagram references

DAY 39

SpecAugment and augmentation ablation

Compute
Modal L4

LEARN

- Time masking.
- Frequency masking.
- Data augmentation as invariance training.

BUILD IN MENDSPEECH

- Add one augmentation intervention to a controlled short run.

EXPERIMENT AND MEASURE

- Compare no augmentation versus selected augmentation with the same seed and training budget.

REQUIRED OUTPUT

- experiments/specaugment_ablation.py
- results/day39_augmentation.csv

> **Completion check:** You can separate the effect of augmentation from the effect of extra training time.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- NVIDIA NeMo ASR training documentation
- RNNT primary references
- Calibration and reliability diagram references

DAY 40

RNNT and streaming decoding

Compute
Modal L4

LEARN

- Encoder.
- Prediction network.
- Joint network.
- Blank handling.
- Streaming emission behavior.
- Difference from CTC independence.

BUILD IN MENDSPEECH

- Run or inspect a FastConformer transducer checkpoint.
- Trace one decoding step conceptually and document tensor roles.

EXPERIMENT AND MEASURE

- Compare CTC and transducer outputs on selected difficult clips.

REQUIRED OUTPUT

- docs/rnnt_walkthrough.md
- results/day40_ctc_vs_rnnt.csv

> **Completion check:** You can explain RNNT without saying only that it is better for streaming.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- NVIDIA NeMo ASR training documentation
- RNNT primary references
- Calibration and reliability diagram references

DAY 41

Confidence calibration for repair decisions

Compute
Modal L4 for logits, local CPU for analysis

LEARN

- Reliability diagrams.
- Expected calibration error intuition.
- Threshold selection from validation data.

BUILD IN MENDSPEECH

- Build a simple calibration analysis for confidence versus correctness.
- Choose policy thresholds on validation, not test.

EXPERIMENT AND MEASURE

- Compare raw and calibrated confidence if a simple method is feasible.

REQUIRED OUTPUT

- src/asr/calibration.py
- results/day41_reliability.png
- configs/repair_modes_calibrated.yaml

Completion check: Repair thresholds are now justified from held out evidence rather than guessed.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- NVIDIA NeMo ASR training documentation
- RNNT primary references
- Calibration and reliability diagram references

DAY 42

Week 6 robustness milestone

Compute
Modal L4

LEARN

- Review fine tuning, augmentation, RNNT, and calibration.

BUILD IN MENDSPEECH

- Switch between base and adapted recognizer in the research console.
- Show clean WER, damaged WER, confidence calibration, and repair percentage.

EXPERIMENT AND MEASURE

- Run one fixed benchmark suite and freeze results for Week 8 comparisons.

REQUIRED OUTPUT

- app/mendspeech_v3_robust.py
- results/week6_frozen_baseline.csv
- reports/week6_training.md

Completion check: MendSpeech can demonstrate measured robustness gains or clearly document a negative result.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- NVIDIA NeMo ASR training documentation
- RNNT primary references
- Calibration and reliability diagram references

Week 7: TTS, Speaker Preservation, and Boundary Matched Reconstruction

> Build MendSpeech V1 as a cascaded selective repair baseline with explicit seam diagnostics.

Day	Focus	Minimum evidence	Compute
Day 43	TTS system anatomy	• Compare several sentences with punctuation and pacing changes.	Modal L4
Day 44	FastSpeech style duration and prosody	• Change speaking rate or duration settings and measure generated length.	Modal L4
Day 45	Vocoder realism and acoustic boundary diagnostics	• Measure inference speed and real time factor. • Create intentionally mismatched generated spans and verify that the boundary diagnostics flag obvious loudness or spectral discontinuities.	Modal L4
Day 46	VITS and end to end synthesis	• Create a listening sheet with randomized sample order.	Modal L4
Day 47	Speaker representation and preservation	• Compare full resynthesis with short span reconstruction for speaker similarity.	Modal L4
Day 48	Selective reconstruction with boundary matched stitching	• Compare full utterance TTS, naive selective repair, and boundary matched selective repair. • Measure preservation percentage, latency, energy discontinuity, and speaker similarity proxy. • Run a small blinded seam audibility check with randomized sample order.	Modal L4 plus local CPU for stitching
Day 49	Week 7 MendSpeech V1 cascaded repair milestone	• Run at least ten cases, including deliberate false repair, missed repair, seam artifacts, and one case where the policy abstains. • Compare naive stitching and boundary matched stitching on the same repaired spans.	Modal L4

DAY 43

TTS system anatomy

Compute
Modal L4

LEARN

- Text or phoneme representation.
- Acoustic model.
- Mel spectrogram or latent representation.
- Vocoder.
- Speaker conditioning.
- Prosody.

BUILD IN MENDSPEECH

- Run a pretrained TTS system on controlled text.
- Save generated waveform and intermediate representations if exposed.

EXPERIMENT AND MEASURE

- Compare several sentences with punctuation and pacing changes.

REQUIRED OUTPUT

- src/tts/baseline.py
- results/day43_tts_samples/
- docs/tts_pipeline.md

Completion check: You can explain how text becomes waveform and where speaker identity can enter.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastSpeech 2 paper
- HiFi GAN paper
- VITS paper
- DSP references for energy matching and equal power crossfades

DAY 44

FastSpeech style duration and prosody

Compute
Modal L4

LEARN

- Duration prediction.
- Pitch and energy predictors.
- Parallel generation intuition.

BUILD IN MENDSPEECH

- Study FastSpeech 2 architecture and inspect an implementation.
- Extract or visualize duration, pitch, or energy controls if available.

EXPERIMENT AND MEASURE

- Change speaking rate or duration settings and measure generated length.

REQUIRED OUTPUT

- docs/day44_fastspeech2.md
- results/day44_prosody_samples/

> **Completion check:** You understand why duration matters when replacing only a short span.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastSpeech 2 paper
- HiFi GAN paper
- VITS paper
- DSP references for energy matching and equal power crossfades

DAY 45

Vocoder realism and acoustic boundary diagnostics

Compute
Modal L4

LEARN

- Mel to waveform generation.
- HiFi GAN style generator and discriminator intuition.
- Phase, bandwidth, and vocoder artifacts.
- Short time energy, local loudness, spectral balance, and room tone as boundary signals.

BUILD IN MENDSPEECH

- Run a neural vocoder or inspect the one used by the selected TTS stack.
- Add boundary diagnostics that measure short time energy and simple spectral statistics before and after a candidate repair span.
- Save a local room tone estimate where possible.

EXPERIMENT AND MEASURE

- Measure inference speed and real time factor.
- Create intentionally mismatched generated spans and verify that the boundary diagnostics flag obvious loudness or spectral discontinuities.

REQUIRED OUTPUT

- results/day45_vocoder_benchmark.csv
- src/repair/boundary_metrics.py
- docs/vocoder_and_boundary_notes.md

> **Completion check:** You can separate acoustic model errors from vocoder artifacts and quantify at least two causes of an audible seam.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastSpeech 2 paper
- HiFi GAN paper
- VITS paper
- DSP references for energy matching and equal power crossfades

DAY 46

VITS and end to end synthesis

Compute
Modal L4

LEARN

- Latent variable modeling.
- Normalizing flow intuition.
- Adversarial training.
- End to end waveform generation.

BUILD IN MENDSPEECH

- Run a VITS style pretrained model or study its code path.
- Compare latency and perceived naturalness with your Week 7 baseline.

EXPERIMENT AND MEASURE

- Create a listening sheet with randomized sample order.

REQUIRED OUTPUT

- results/day46_tts_comparison.csv
- results/day46_listening_sheet.md

> **Completion check:** You can explain why different TTS architectures behave differently for selective repair.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastSpeech 2 paper
- HiFi GAN paper
- VITS paper
- DSP references for energy matching and equal power crossfades

DAY 47

Speaker representation and preservation

Compute
Modal L4

LEARN

- Speaker embeddings.
- Reference conditioned synthesis.
- Speaker similarity as a measurable but imperfect proxy.
- Consent and voice identity boundaries.

BUILD IN MENDSPEECH

- Choose a speaker conditioned or reference conditioned path that is legally and ethically appropriate for your own or consented samples.
- Compute speaker embeddings before and after synthesis if tooling is available.

EXPERIMENT AND MEASURE

- Compare full resynthesis with short span reconstruction for speaker similarity.

REQUIRED OUTPUT

- src/tts/speaker_conditioning.py
- results/day47_speaker_similarity.csv
- docs/voice_use_policy.md

> **Completion check:** You can discuss speaker similarity measurements and their limitations without claiming identity preservation from listening alone.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastSpeech 2 paper
- HiFi GAN paper
- VITS paper
- DSP references for energy matching and equal power crossfades

DAY 48

Selective reconstruction with boundary matched stitching

Compute
Modal L4 plus local CPU for stitching

LEARN

- Repair span text selection.
- Timing constraints and duration control.
- Boundary padding and silence handling.
- Short time energy matching and local loudness matching.
- Linear versus equal power crossfades.
- Spectral and room tone mismatch.
- Why ASR to text to TTS can lose pitch, emotion, breathing, and coarticulation.

BUILD IN MENDSPEECH

- Take a known damaged interval and synthesize only its transcript span.
- Match generated duration to the target interval without changing untouched speech.
- Match local energy before stitching and implement both linear and equal power crossfades.
- Add optional room tone under the regenerated span when the original context supports it.
- Log preserved samples, reconstructed samples, boundary length, and all matching parameters.

EXPERIMENT AND MEASURE

- Compare full utterance TTS, naive selective repair, and boundary matched selective repair.
- Measure preservation percentage, latency, energy discontinuity, and speaker similarity proxy.
- Run a small blinded seam audibility check with randomized sample order.

REQUIRED OUTPUT

- src/repair/reconstruct.py
- src/repair/stitch.py
- src/repair/boundary_metrics.py
- results/day48_selective_samples/
- results/day48_seam_ablation.csv

> **Completion check:** The final audio keeps most original samples, replaces only a targeted interval, and shows measurably smoother boundaries than naive stitching.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastSpeech 2 paper
- HiFi GAN paper
- VITS paper
- DSP references for energy matching and equal power crossfades

DAY 49

Week 7 MendSpeech V1 cascaded repair milestone

Compute
Modal L4

LEARN

- Review TTS, duration, vocoder behavior, speaker conditioning, boundary matching, and information lost through the text bottleneck.
- Treat the cascaded path as a real time baseline, not as the final frontier of speech restoration.

BUILD IN MENDSPEECH

- Pipeline: damaged audio to streaming ASR to uncertain span to policy decision to speaker conditioned reconstruction to boundary matched waveform.
- Show preserved and reconstructed intervals with distinct visualization.
- Add a V1 label in results so the Week 8 direct audio repair comparison is explicit.

EXPERIMENT AND MEASURE

- Run at least ten cases, including deliberate false repair, missed repair, seam artifacts, and one case where the policy abstains.
- Compare naive stitching and boundary matched stitching on the same repaired spans.

REQUIRED OUTPUT

- app/mendspeech_v4_cascaded.py
- demos/week7_before_after/
- results/week7_stitching_ablation.csv
- reports/week7_cascaded_repair.md

> **Completion check:** MendSpeech V1 is a measured cascaded baseline whose strengths and prosody or seam limitations are documented rather than hidden.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- FastSpeech 2 paper
- HiFi GAN paper
- VITS paper
- DSP references for energy matching and equal power crossfades

Week 8: Research Capstone: Cascaded Versus Direct Repair

> Freeze the benchmark, run controlled ablations, compare architectures, and publish a reproducible result.

Day	Focus	Minimum evidence	Compute
Day 50	Freeze research questions and baselines	• Run a tiny dry run to ensure every result field is populated.	Local CPU for planning, Modal L4 for dry run
Day 51	Release SpeechDamageBench v1 and freeze evaluation	• Reinstall the package in a clean environment. • Regenerate a sample from the manifest and verify its checksum. • Validate that clean references remain unchanged.	Local CPU
Day 52	Run recognition and context ablations	• Plot WER versus latency and mark Pareto efficient points.	Modal L4, keep hardware fixed
Day 53	Run cascaded repair and seam ablations	• Test whether repairing more audio always helps intelligibility. • Test whether boundary matching reduces seam artifacts without materially increasing latency. • Keep recognition outputs fixed for the stitching comparison so only the repair method changes.	Modal L4
Day 54	Compare against direct latent or codec audio inpainting	• Compare raw damaged audio, MendSpeech V1 cascaded repair, full resynthesis, and the direct audio baseline on the same cases. • Select at least ten worst or most revealing cases and inspect them manually. • Identify at least one regime where each approach has an advantage, or explicitly report if the data does not support that conclusion.	Modal L4 or the smallest GPU that can run the chosen pretrained baseline, plus local analysis
Day 55	Write the research report and reproducibility guide	• Audit every major claim against a concrete table, figure, or experiment result. • Remove or soften any conclusion that is not directly supported by frozen evidence. • Verify that the report distinguishes measured facts from hypotheses and future work.	Local CPU
Day 56	Final product, demo, and clean reproduction	• Record a concise demo and create a final architecture diagram. • Reproduce one benchmark end to end from the documented command. • Verify that every public chart can be regenerated from saved result files.	Modal L4 for inference, local CPU for interface and analysis

DAY 50

Freeze research questions and baselines

Compute
Local CPU for planning, Modal L4 for dry run

LEARN

- Primary question: can selective semantic repair improve intelligibility while preserving more original speech than full resynthesis?
- Secondary question: can uncertainty guided context allocation improve the latency versus accuracy operating point?
- Architecture question: when does cascaded ASR plus TTS repair beat or lose to a pretrained direct latent or codec audio inpainting baseline?
- Define null outcomes, failure criteria, and claims you will not make.

BUILD IN MENDSPEECH

- Freeze code revision, model revisions, datasets, hardware, corruption configs, and metrics.
- Define baselines: raw damaged audio, full resynthesis, MendSpeech V1 cascaded repair, fixed context, adaptive context, and one pretrained direct audio inpainting baseline if reproducible.
- Do not train the direct inpainting model from scratch. The purpose is architectural comparison, not a second major training project.

EXPERIMENT AND MEASURE

- Run a tiny dry run to ensure every result field is populated.

REQUIRED OUTPUT

- experiments/capstone_protocol.md
- configs/capstone_frozen.yaml
- docs/baseline_definitions.md

> **Completion check:** Another engineer could reproduce the protocol and understand exactly which claims compare cascaded repair with direct audio repair.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Your frozen protocol and prior results
- A reproducible pretrained direct latent or codec audio inpainting baseline
- Primary papers only when needed to interpret a result

DAY 51

Release SpeechDamageBench v1 and freeze evaluation

Compute
Local CPU

LEARN

- Severity grids.
- Speaker separated evaluation.
- Seed control and deterministic manifests.
- Package versioning and reproducibility.
- Clean regression cases that must remain untouched.

BUILD IN MENDSPEECH

- Finalize the independent SpeechDamageBench package with noise, clipping, bandwidth, dropout, and reverberation presets.
- Generate the frozen test matrix and lock manifest checksums.
- Add an installation command and a one command example that reproduces one benchmark item.

EXPERIMENT AND MEASURE

- Reinstall the package in a clean environment.
- Regenerate a sample from the manifest and verify its checksum.
- Validate that clean references remain unchanged.

REQUIRED OUTPUT

- speechdamagebench/
- speechdamagebench/README.md
- speechdamagebench/CHANGELOG.md
- benchmarks/speechdamagebench_manifest.csv
- benchmarks/README.md

Completion check: SpeechDamageBench is independently installable, deterministic, versioned, and usable without MendSpeech.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Your frozen protocol and prior results
- A reproducible pretrained direct latent or codec audio inpainting baseline
- Primary papers only when needed to interpret a result

DAY 52

Run recognition and context ablations

Compute
Modal L4, keep hardware fixed

LEARN

- Fixed lookahead comparison.
- Adaptive context policy.
- WER, latency, RTF, memory, confidence behavior.

BUILD IN MENDSPEECH

- Run every streaming condition on the exact same benchmark subset.
- Repeat timing runs enough to estimate variance.
- Record GPU type and environment automatically through the Modal runner.

EXPERIMENT AND MEASURE

- Plot WER versus latency and mark Pareto efficient points.

REQUIRED OUTPUT

- results/capstone_streaming.csv
- results/streaming_pareto.png

> **Completion check:** You can say whether adaptive context helped, hurt, or made no meaningful difference.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Your frozen protocol and prior results
- A reproducible pretrained direct latent or codec audio inpainting baseline
- Primary papers only when needed to interpret a result

DAY 53

Run cascaded repair and seam ablations

Compute
Modal L4

LEARN

- Repair threshold.
- Repair span padding.
- Preserve percentage.
- Full resynthesis baseline.
- Boundary energy matching, crossfade choice, and seam artifact rate.

BUILD IN MENDSPEECH

- Run Preserve, Balanced, Rescue, full resynthesis, naive selective stitching, and boundary matched selective stitching.
- Record original waveform retained, repair percentage, end to end latency, speaker similarity proxy, and seam metrics.

EXPERIMENT AND MEASURE

- Test whether repairing more audio always helps intelligibility.
- Test whether boundary matching reduces seam artifacts without materially increasing latency.
- Keep recognition outputs fixed for the stitching comparison so only the repair method changes.

REQUIRED OUTPUT

- results/capstone_cascaded_repair.csv
- results/repair_tradeoff.png
- results/seam_ablation.png

> **Completion check:** You have a defensible result for the cascaded selective repair path and can separate recognition, reconstruction, and stitching effects.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Your frozen protocol and prior results
- A reproducible pretrained direct latent or codec audio inpainting baseline
- Primary papers only when needed to interpret a result

DAY 54

Compare against direct latent or codec audio inpainting

Compute
Modal L4 or the smallest GPU that can run the chosen pretrained baseline, plus local analysis

LEARN

- Why text is an information bottleneck for prosody and acoustic continuity.
- Direct audio inpainting in latent or codec token spaces at a conceptual level.
- Fair baseline design when systems have different latency and compute profiles.
- Failure taxonomy across semantic correctness, speaker similarity, prosody, seam quality, and compute.

BUILD IN MENDSPEECH

- Select one reproducible pretrained direct audio inpainting or restoration baseline.
- Wrap it behind the same benchmark interface used by MendSpeech.
- Feed identical SpeechDamageBench cases and record the same metrics wherever they are meaningful.
- Create a failure casebook covering both architectures.
- Keep abstention active for MendSpeech when the inferred content is not reliable enough to reconstruct safely.

EXPERIMENT AND MEASURE

- Compare raw damaged audio, MendSpeech V1 cascaded repair, full resynthesis, and the direct audio baseline on the same cases.
- Select at least ten worst or most revealing cases and inspect them manually.
- Identify at least one regime where each approach has an advantage, or explicitly report if the data does not support that conclusion.

REQUIRED OUTPUT

- src/baselines/direct_audio_inpaint.py
- results/capstone_architecture_compare.csv
- results/capstone_failure_casebook.md
- results/architecture_tradeoff.png
- src/controller/abstain.py

Completion check: You can explain when the cascaded path is competitive, where it loses acoustic information, and whether direct audio repair earns its extra complexity on your benchmark.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Your frozen protocol and prior results
- A reproducible pretrained direct latent or codec audio inpainting baseline
- Primary papers only when needed to interpret a result

DAY 55

Write the research report and reproducibility guide

Compute
Local CPU

LEARN

- Abstract, motivation, hypotheses, method, baselines, metrics, results, limitations, ethics, and future work.
- Difference between observation and causal claim.
- How to report a negative or mixed architectural comparison honestly.

BUILD IN MENDSPEECH

- Write the complete report.
- Add exact reproduction commands and environment capture.
- Include the cascaded versus direct repair comparison as a dedicated section.
- Document seam limitations, prosody loss, and any conditions where the direct baseline is clearly stronger.
- Include plots with captions that state what changed and what stayed fixed.

EXPERIMENT AND MEASURE

- Audit every major claim against a concrete table, figure, or experiment result.
- Remove or soften any conclusion that is not directly supported by frozen evidence.
- Verify that the report distinguishes measured facts from hypotheses and future work.

REQUIRED OUTPUT

- REPORT.md
- REPRODUCE.md
- results/final_figures/
- docs/limitations_and_claims.md

> **Completion check:** A technical reader can understand the contribution, the architectural tradeoff, and the limitations without opening the source code first.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Your frozen protocol and prior results
- A reproducible pretrained direct latent or codec audio inpainting baseline
- Primary papers only when needed to interpret a result

DAY 56

Final product, demo, and clean reproduction

Compute
Modal L4 for inference, local CPU for interface and analysis

LEARN

- Review the complete path from waveform and controlled corruption to streaming encoder, uncertainty, repair policy, cascaded reconstruction, direct audio baseline, and evaluation.

BUILD IN MENDSPEECH

- Build the final demo with upload or microphone input, controlled damage, live transcript, uncertainty heatmap, preserved versus repaired timeline, before and after playback, metrics, and architecture selection for benchmark playback.
- Show which milliseconds were preserved, reconstructed by the cascaded path, or repaired by the direct baseline.
- Reproduce one frozen benchmark from a fresh environment and tag a stable release.

EXPERIMENT AND MEASURE

- Record a concise demo and create a final architecture diagram.
- Reproduce one benchmark end to end from the documented command.
- Verify that every public chart can be regenerated from saved result files.

REQUIRED OUTPUT

- app/mendspeech_final.py
- README.md
- demos/final_demo.mp4
- docs/architecture.png
- release_notes.md
- results/reproduction_check.txt

> **Completion check:** A new user can understand, run, and evaluate MendSpeech, SpeechDamageBench, the cascaded baseline, and the direct audio comparison, and you can defend every major design decision.

Study method
25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Your frozen protocol and prior results
- A reproducible pretrained direct latent or codec audio inpainting baseline
- Primary papers only when needed to interpret a result